

# Batch Discounts and Circuit Breakers, Worked Out by Hand

Two cost levers, two clean formulas: deferring 40% of work to a 50%-off batch endpoint saves 20%, and a \$0.30 ceiling halves a heavy-tailed agent's mean spend.

**What this document does.** It takes two production cost levers from Module 4.3 — the *batch API* (offline work at ~50% off) and the *per-trajectory circuit breaker* (a dollar ceiling that kills runaway runs) — and works each one to a number on a tiny, hand-sized example. For the batch lever we defer a fraction  $f$  of the refund-resolver's workload and read off the saving. For the breaker we take a ten-run cost sample with one fat tail and apply the module's actual \$0.30 ceiling, computing the mean before and after. We close with the burn-rate alert that watches a budget in real time. Every number is reproduced by the Python program in Appendix A.

**Where this sits.** This is **Topic 2 of Module 4.3**. Its companions: Module 4.3 Topic 1 (the model cascade, whose blended \$0.022 is the bill these levers further bend) and Module 4.2 (per-trajectory cost tags — the measurements that tell the breaker when to trip and the burn-rate alert when to page).

**Concepts covered, in order:** batch deferral as a linear discount  $c(1 - \frac{1}{2}f)$  · saving vs. deferred fraction · the circuit breaker as  $\mathbb{E}[\min(\text{cost}, K)]$  · clipping a heavy tail · mean before/after and percent saved · burn rate =  $\text{spend}/(B \cdot \text{elapsed}/T)$  · when each lever helps and what it trades away.

*Internal learning material. Numbers are reproducible from Appendix A. v1.0*

---

## 1 The setup: two levers, each a one-line formula

Module 4.3 stacks seven runtime levers on top of caching. Two of them are pure arithmetic once you fix the right quantity, and those are the two we work here.

| Quantity                                 | Symbol | Value here          |
|------------------------------------------|--------|---------------------|
| Sync (list) cost of the workload         | $c$    | \$1.00 (normalised) |
| Batch discount off list                  | $d$    | 0.50 (50% off)      |
| Fraction of workload deferrable to batch | $f$    | 0.40                |
| Per-trajectory cost ceiling (breaker)    | $K$    | \$0.30              |
| Budget over the alerting window          | $B$    | \$1,000             |
| Window length                            | $T$    | 24 h                |

**Notation.** For the batch lever we normalise the sync cost to  $c = \$1$  so every result reads as a fraction of list price. For the breaker we work an explicit ten-run cost sample. For the burn-rate alert we track dollars spent against an elapsed fraction of the window. We carry three decimals throughout.

**Intuition**

These two levers pull in opposite directions on the same dial. Batching *lowers the price* of work you are happy to wait for; the circuit breaker *caps the quantity* of work any single runaway can buy. One is a planned discount, the other an emergency brake — and a healthy fleet runs both at once.

**2 Step 1 — batch deferral is a linear discount**

A batch endpoint processes requests within hours instead of seconds, at roughly half price. If you can defer a fraction  $f$  of your workload to it, that fraction is billed at  $(1-d)c$  while the remaining  $(1-f)$  stays at full sync price  $c$ . The effective cost per unit of workload is

$$c_{\text{eff}}(f) = (1-f)c + f(1-d)c = c(1-df). \quad (1)$$

With the module's  $d = 0.5$  this collapses to the clean half-discount form

$$c_{\text{eff}}(f) = c\left(1 - \frac{1}{2}f\right) \quad (2)$$

and the saving is exactly  $\frac{1}{2}f$  of list. Working the module's  $f = 0.4$ :  $c_{\text{eff}} = c(1 - 0.5 \times 0.4) = c(1 - 0.20) = 0.80c$ . Deferring 40% of the workload to the 50%-off batch endpoint cuts the bill by **20%**. Tabulating against  $f$ :

| deferred fraction $f$ | $c_{\text{eff}}$ | saving | note                          |
|-----------------------|------------------|--------|-------------------------------|
| 0.0                   | 1.00 $c$         | 0%     | all sync                      |
| 0.2                   | 0.90 $c$         | 10%    |                               |
| 0.4                   | 0.80 $c$         | 20%    | worked example                |
| 0.6                   | 0.70 $c$         | 30%    |                               |
| 0.8                   | 0.60 $c$         | 40%    |                               |
| 1.0                   | 0.50 $c$         | 50%    | fully offline → full discount |

Check,  $f = 0.4$ :  $1 - 0.5(0.4) = 1 - 0.2 = 0.80$ , so saving = 20%. ✓ The saving is a straight line through the origin with slope  $\frac{1}{2}$ : each 10 points of workload you can move to batch buys 5 points off the bill, all the way to the 50% ceiling at  $f = 1$ .

**Mini-example — this operation in isolation**

The cap on this lever is the discount itself. No matter how much you defer, you can never beat 50% off, because the deferred work still costs half. So the lever's value is bounded by how much of your workload *tolerates hours of latency*: evals, judge sweeps, embedding backfills, nightly summaries. The interactive refund-resolver itself cannot be deferred — but its offline eval suite, running the same model, can. The question is never “how cheap can batch make this?” but “what fraction of my spend is genuinely offline?”

### 3 Step 2 — the circuit breaker clips the tail

Per-trajectory cost is heavy-tailed: most runs are cheap, a few runaways are not. A circuit breaker caps the spend on any single run at a ceiling  $K$  — the moment a run’s accumulated cost crosses  $K$ , it is killed and handed off. The expected spend per run with the breaker is the clipped expectation

$$\mathbb{E}[\min(\text{cost}, K)] = \frac{1}{n} \sum_{j=1}^n \min(x_j, K), \quad (3)$$

estimated over a sample of run costs  $x_1, \dots, x_n$ . Take a ten-run sample with one fat tail (dollars per trajectory), and the module’s ceiling  $K = \$0.30$ :

|                   |      |      |      |      |      |      |      |             |             |             |
|-------------------|------|------|------|------|------|------|------|-------------|-------------|-------------|
| run $x_j$         | 0.05 | 0.08 | 0.12 | 0.15 | 0.18 | 0.22 | 0.30 | <b>0.45</b> | <b>0.90</b> | <b>1.55</b> |
| $\min(x_j, 0.30)$ | 0.05 | 0.08 | 0.12 | 0.15 | 0.18 | 0.22 | 0.30 | 0.30        | 0.30        | 0.30        |

The three bold runs (0.45, 0.90, 1.55) exceed the ceiling and are clipped to \$0.30. Summing each row and dividing by  $n = 10$ :

$$\text{mean before} = \frac{4.00}{10} = \$0.4000, \quad \text{mean after} = \frac{2.00}{10} = \$0.2000.$$

Check the raw sum:

$$0.05 + 0.08 + 0.12 + 0.15 + 0.18 + 0.22 + 0.30 + 0.45 + 0.90 + 1.55 = 4.00,$$

so the mean is \$0.40. ✓ For the clipped sum, the seven runs at or below  $K$  sum to  $0.05 + 0.08 + 0.12 + 0.15 + 0.18 + 0.22 + 0.30 = 1.10$ , and the three clipped runs each contribute \$0.30, adding 0.90: total  $1.10 + 0.90 = 2.00$ , mean \$0.20. ✓ The breaker

$$\boxed{\text{saved} = \frac{0.40 - 0.20}{0.40} = 50\%} \quad (4)$$

by killing just **3 of 10** runs. Almost all the damage was in the single \$1.55 outlier; clipping it alone removed  $\$1.25/10 = \$0.125$  per run, more than half the total saving.

#### Watch out

The breaker bounds your *worst case*, not your *typical* case. Here it slashed the mean only because the tail was monstrous — in a fleet where every run is well under \$0.30 the breaker saves nothing and trips never. That is the healthy state: the module’s production breaker sits at limit \$0.30 with p99 \$0.18, tripping about twice an hour. If your breaker starts saving 50% of your mean in steady state, the breaker is not the win — it is the alarm. Something upstream (a routing bug, a tool loop) is generating runaways you should fix, not merely clip.

#### Intuition

A cost ceiling converts an *unbounded* tail risk into a *bounded, known* one. Without it, one pathological trajectory — a tool that keeps returning “try again,” a loop that never calls `final_answer` — can bill arbitrarily much before anyone notices. With it, the most any

single run can cost is  $K$ , full stop. You trade a few prematurely-killed runs (escalated to a human) for a hard cap on the blast radius of the worst one.

## 4 Step 3 — the burn-rate alert

Breakers stop one run; the burn-rate alert watches the *whole budget* in flight. Give a budget  $B$  to spend over a window of length  $T$ . At an elapsed time  $t$  you “should” have spent, on a flat pace,  $B \cdot t/T$ . The burn rate is actual spend over that expected pace:

$$\text{burn} = \frac{\text{spend so far}}{B \cdot (\text{elapsed}/T)}. \quad (5)$$

A burn rate above 1 means you are spending faster than the flat budget allows — on track to blow it before the window closes. Worked example: budget  $B = \$1,000$  over  $T = 24\text{h}$ ; after 6 h you have spent \$320.

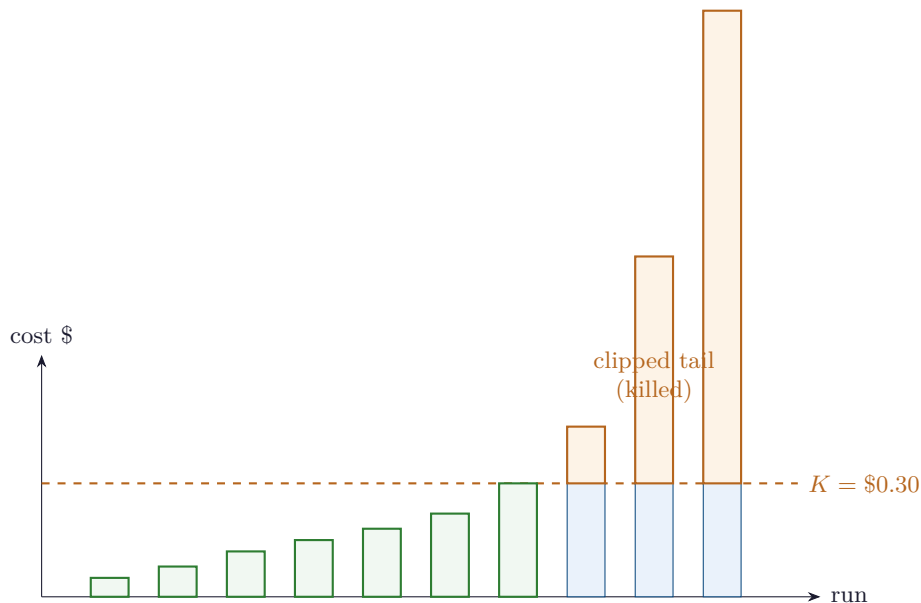
$$\text{expected by now} = 1000 \times \frac{6}{24} = \$250, \quad \text{burn} = \frac{320}{250} = 1.28.$$

Check:  $\$1000 \times 0.25 = \$250$ ;  $320/250 = 1.28$ . ✓ A burn rate of **1.28** ( $> 1$ ) fires the alert. Extrapolating the current pace to the end of the window forecasts spend  $\times T/\text{elapsed} = 320 \times 24/6 = \$1,280$  — a 28% overrun, exactly as the burn rate said.

### Mini-example — this operation in isolation

The burn rate is a unit-free early-warning gauge: 1.0 is dead on pace, 0.8 is comfortably under, 1.28 is heading for a 28% overshoot. Because it normalises by elapsed time, it fires *early* — four hours into a 24-hour window, long before the raw spend looks alarming. That head start is the whole point: it pages the on-call while there is still budget left to protect.

## 5 The breaker, drawn



The seven green bars sit under the ceiling and pay in full. The three tall bars pierce  $K = \$0.30$ ; everything above the dashed line (the orange caps) is the spend the breaker *refuses*. Mean cost falls from \$0.40 to \$0.20 because that orange area is removed from the average. The picture is Equation (3).

## 6 Cost cheat-sheet

|                         |                                                                                     |
|-------------------------|-------------------------------------------------------------------------------------|
| Batch effective cost    | $c_{\text{eff}}(f) = c(1 - df)$                                                     |
| at $d = 0.5$            | $c_{\text{eff}}(f) = c(1 - \frac{1}{2}f)$                                           |
| Batch saving            | $df$ ( $= \frac{1}{2}f$ at 50% off)                                                 |
| Breaker expected spend  | $\mathbb{E}[\min(\text{cost}, K)] = \frac{1}{n} \sum_j \min(x_j, K)$                |
| Breaker % saved         | $(\text{mean}_{\text{raw}} - \text{mean}_{\text{clip}}) / \text{mean}_{\text{raw}}$ |
| Burn rate               | $\text{burn} = \text{spend} / (B \cdot \text{elapsed} / T)$                         |
| Projected end-of-window | $\text{spend} \cdot T / \text{elapsed}$                                             |
| Alert condition         | $\text{burn} > 1$                                                                   |

## 7 An honest word about these numbers

The discount ( $d = 0.5$ ), the deferred fraction ( $f = 0.4$ ), the ceiling ( $K = \$0.30$ ), and the ten-run sample are illustrative round numbers, chosen so the means land on \$0.40 and \$0.20 and the saving reads as a clean 50%. Real batch discounts are provider-dependent and “~50%”; real run-cost distributions are messier than ten tidy points and their clipped saving depends entirely on how fat your tail actually is; real budgets and windows are whatever finance sets. What is *not* illustrative is the structure: batch deferral is the linear discount  $c(1 - df)$ ; a cost ceiling yields the clipped expectation  $\mathbb{E}[\min(\text{cost}, K)]$  which can only lower the mean; and the burn rate is spend over expected-pace, firing above 1. Change the constants and the dollar figures move; these three shapes do not. That is what to carry forward — the levers are levers because their formulas are simple and monotone.

## Appendix A — Reference implementation

Every number in this document is produced by the program below. Run it to reproduce the batch table, the breaker means, and the burn-rate example; change `d`, `sample`, `K`, or the budget to explore your own fleet’s economics.

```

1 # batch_breaker.py -- reproduces every number in
2 # "Batch Discounts and Circuit Breakers".
3
4 # --- batch deferral: c_eff = c*(1 - d*f) ---
5 c, d = 1.0, 0.5 # sync cost (normalised), batch discount
6 print("=== batch deferral (d=0.5) ===")
7 for f in (0.0, 0.2, 0.4, 0.6, 0.8, 1.0):
8     eff = c * (1 - d * f)
9     print(f"f={f:.1f} c_eff={eff:.2f} c saving={(1-eff)*100:.0f}%")
10
11 # --- circuit breaker: E[min(cost, K)] ---
12 sample = [0.05, 0.08, 0.12, 0.15, 0.18, 0.22, 0.30, 0.45, 0.90, 1.55]
```

```

13 K = 0.30
14 n = len(sample)
15 mean_raw = sum(sample) / n
16 mean_clip = sum(min(x, K) for x in sample) / n
17 killed = [x for x in sample if x > K]
18 print("=== circuit breaker (K=$0.30) ===")
19 print(f"mean before = ${mean_raw:.4f}")
20 print(f"mean after = ${mean_clip:.4f}")
21 print(f"saved = {(mean_raw-mean_clip)/mean_raw*100:.1f}% "
22       f"(killed {len(killed)} of {n})")
23
24 # --- burn-rate alert: spend / (B * elapsed/T) ---
25 B, T, elapsed, spend = 1000.0, 24.0, 6.0, 320.0
26 expected = B * elapsed / T
27 burn = spend / expected
28 projected = spend * T / elapsed
29 print("=== burn-rate alert ===")
30 print(f"expected by now = ${expected:.2f}")
31 print(f"burn rate = {burn:.2f} (>1 means over budget pace: {burn > 1})")
32 print(f"projected end-of-window = ${projected:.2f}")
33
34 # Expected output:
35 # === batch deferral (d=0.5) ===
36 # f=0.0 c_eff=1.00c saving=0%
37 # f=0.2 c_eff=0.90c saving=10%
38 # f=0.4 c_eff=0.80c saving=20%
39 # f=0.6 c_eff=0.70c saving=30%
40 # f=0.8 c_eff=0.60c saving=40%
41 # f=1.0 c_eff=0.50c saving=50%
42 # === circuit breaker (K=$0.30) ===
43 # mean before = $0.4000
44 # mean after = $0.2000
45 # saved = 50.0% (killed 3 of 10)
46 # === burn-rate alert ===
47 # expected by now = $250.00
48 # burn rate = 1.28 (>1 means over budget pace: True)
49 # projected end-of-window = $1280.00

```

— end of document —